

School of Computer Science and Artificial Intelligence

Lab Assignment # 12

Name of Student : B. Sai Rithvik
Enrollment No. : 2303A51532
Batch No. 22

Task Description -1 (Data Structures – Stack Implementation with AI Assistance)

➤ Task: Use AI assistance to generate a Python program that implements a Stack data structure.

Instructions: Prompt AI to create a Stack class with the following methods:

- push(element), pop(), peek(), is_empty()
- Ensure proper error handling for stack underflow.
- Ask AI to include clear docstrings for each method.

Expected Output:

- A functional Python program implementing a Stack using a class.
- Properly documented methods with docstrings.

Prompt:

generate a python program to implement a Stack class with push, pop, peek, is Empty and size methods.and ensure that proper error handling for stack underflow and overflow conditions and include a docstring for each method to explain its functionality.

Code:

```
Assignment12.py X
Lab > Assignment12.py > Stack > push
1 # Task-1
2 # generate a python program to implement a Stack class with push, pop, peek, isEmpty and size methods.and ensure that proper error handling for stack underflow and overflow conditions and include a docstring for each method to explain its functionality.
3 class Stack:
4     def __init__(self, max_size=10):
5         self.items = []
6         self.max_size = max_size
7
8     def push(self, item):
9         """Push an item onto the stack."""
10        if len(self.items) >= self.max_size:
11            raise OverflowError("Stack overflow: Cannot push to a full stack.")
12        self.items.append(item)
13
14    def pop(self):
15        """Pop an item from the stack."""
16        if self.isEmpty():
17            raise IndexError("Stack underflow: Cannot pop from an empty stack.")
18        return self.items.pop()
19
20    def peek(self):
21        """Return the top item of the stack without removing it."""
22        if self.isEmpty():
23            raise IndexError("Stack is empty: Cannot peek at an empty stack.")
24        return self.items[-1]
25
26    def isEmpty(self):
27        """Check if the stack is empty."""
28        return len(self.items) == 0
29
30    def size(self):
31        """Return the number of items in the stack."""
32        return len(self.items)
33
34    # Example usage:
35    if __name__ == "__main__":
36        stack = Stack(max_size=5)
37        stack.push(1)
38        stack.push(2)
39        stack.push(3)
40        print("Top item:", stack.peek()) # Output: Top item: 3
41        print("Stack size:", stack.size()) # Output: Stack size: 3
42        print("Popped item:", stack.pop()) # Output: Popped item: 3
43        print("Is stack empty?", stack.isEmpty()) # Output: Is stack empty? False
44        stack.pop()
45        print("Is stack empty?", stack.isEmpty()) # Output: Is stack empty? True
46        try:
47            stack.pop() # This will raise an error
48        except IndexError as e:
49            print(e) # Output: Stack underflow: Cannot pop from an empty stack.
50        try:
51            stack.peek() # This will raise an error
52        except IndexError as e:
53            print(e) # Output: Stack is empty: Cannot peek at an empty stack.
54        try:
55            for i in range(6):
56                stack.push(i) # This will raise an error on the 6th push
57        except OverflowError as e:
58            print(e) # Output: Stack overflow: Cannot push to a full stack.
59
60
```

Output:

```
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC> python AI_AC/Lab/Assignment12.py
Top item: 3
Stack size: 3
Popped item: 3
Is stack empty? False
Is stack empty? True
Stack underflow: Cannot pop from an empty stack.
Stack is empty: Cannot peek at an empty stack.
Stack overflow: Cannot push to a full stack.
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC> █
```

Justification:

The Stack class is implemented with methods for pushing, popping, peeking, checking if the stack is empty, and getting the size of the stack. Each method includes error handling for stack overflow and underflow conditions. The example usage demonstrates how to use the Stack class and handles edge cases effectively. The time complexity for push and pop operations is $O(1)$, while peek, is Empty, and size operations also have a time complexity of $O(1)$. The space complexity is $O(n)$ where n is the number of items in the stack.

Task Description -2 (Algorithms – Linear vs Binary Search Analysis)

➤ **Task:** Use AI to implement and compare Linear Search and Binary Search algorithms in Python.

Instructions:

- Prompt AI to generate:
- `Linear_search(arr, target)`
- `Binary_search(arr, target)`
- Include docstrings explaining:
- Working principle
- Test both algorithms using different input sizes.

Expected Output:

- Python implementations of both search algorithms.
- AI-generated comments and complexity analysis.
- Test results showing correctness and comparison.

Prompt:-

generate a python program to implement a compare different between Linear Search and Binary Search algorithms. The program should include functions for both search methods, and it should allow to test both algorithms using different input sizes and measure their

execution time. Additionally, the program should handle edge cases, such as searching for an element that is not present in the list and include docstring for each function.

Code:

```

Assignment12.py X
Lab > Assignment12.py > ...
61
62 # Task:2
63 # generate a python program to implement a compare different between Linear Search and Binary Search algorithms. The program should include functions for both search methods, and it should allow to test both algor
64 import time
65 def linear_search(arr, target):
66     """Perform linear search to find the target in the array."""
67     for index, value in enumerate(arr):
68         if value == target:
69             return index
70     return -1 # Target not found
71 def binary_search(arr, target):
72     """Perform binary search to find the target in the sorted array."""
73     left, right = 0, len(arr) - 1
74     while left <= right:
75         mid = left + (right - left) // 2
76         if arr[mid] == target:
77             return mid
78         elif arr[mid] < target:
79             left = mid + 1
80         else:
81             right = mid - 1
82     return -1 # Target not found
83 # Example usage:
84 if __name__ == "__main__":
85     # Test data
86     arr = list(range(1, 10001)) # Sorted array from 1 to 10000
87     target = 9999
88
89     # Test Linear Search
90     start_time = time.time()
91     result_linear = linear_search(arr, target)
92     end_time = time.time()
93     print(f"Linear Search: Target found at index {result_linear}, Time taken: {end_time - start_time:.6f} seconds")
94
95     # Test Binary Search
96     start_time = time.time()
97     result_binary = binary_search(arr, target)
98     end_time = time.time()
99     print(f"Binary Search: Target found at index {result_binary}, Time taken: {end_time - start_time:.6f} seconds")
100
101     # Edge case: Searching for an element that is not present
102     target_not_found = 10001
103     print("\nTesting edge case for element not present:")
104
105     # Linear Search for element not present
106     start_time = time.time()
107     result_linear_not_found = linear_search(arr, target_not_found)
108     end_time = time.time()
109     print(f"Linear Search (not found): Result {result_linear_not_found}, Time taken: {end_time - start_time:.6f} seconds")
110
111     # Binary Search for element not present
112     start_time = time.time()
113     result_binary_not_found = binary_search(arr, target_not_found)
114     end_time = time.time()
115     print(f"Binary Search (not found): Result {result_binary_not_found}, Time taken: {end_time - start_time:.6f} seconds")
116
117

```

Output:

```

Assignment12.py
Linear Search: Target found at index 9998, Time taken: 0.000298 seconds
Binary Search: Target found at index 9998, Time taken: 0.000014 seconds

Testing edge case for element not present:
Linear Search (not found): Result -1, Time taken: 0.000360 seconds
Binary Search (not found): Result -1, Time taken: 0.000006 seconds
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC>

```

Justification:

The program implements both linear search and binary search algorithms, allowing us to compare their performance with different input sizes. The time complexity of linear search is $O(n)$, while the time complexity of binary search is $O(\log n)$. The program also handles edge cases effectively, such as searching for an element that is not present in the list. Each function includes a docstring explaining its functionality, and the example usage demonstrates how to use both search methods and measure their execution time.

Task Description -3 (Test Driven Development – Simple Calculator Function)

➤ Task:

Apply Test Driven Development (TDD) using AI assistance to develop a calculator function.

Instructions:

- Prompt AI to first generate unit test cases for addition and subtraction.
- Run the tests and observe failures.
- Ask AI to implement the calculator functions to pass all tests.
- Re-run the tests to confirm success.

Expected Output:

- **Separate test file and implementation file.**
- **Test cases executed before implementation.**

Prompt:

generate a python program to implement a calculator function , unit tests for addition and subtraction operations. Do not provide the implementation yet, as I need to run these tests and watch them fail first to follow TDD principles.

Code:

```

Assignment12.py
Lab > Assignment12.py > TestCalculator > test_subtraction

116
117
118 '''
119 # Task-3
120 # generate a python program to implement a calculator function , unit tests for addition and subtraction operations. Do not provide the implementation yet, as I need to
121 import unittest
122 def calculator(operation, a, b):
123     """Perform basic arithmetic operations based on the given operation."""
124     if operation == 'add':
125         return a + b
126     elif operation == 'subtract':
127         return a - b
128     else:
129         raise ValueError("Unsupported operation. Use 'add' or 'subtract'.")
130 class TestCalculator(unittest.TestCase):
131     def test_addition(self):
132         """Test addition operation."""
133         self.assertEqual(calculator('add', 2, 3), 5) # This will fail
134         self.assertEqual(calculator('add', 0, 0), 0)
135         self.assertEqual(calculator('add', -1, 1), 0)
136         self.assertEqual(calculator('add', -1, -1), -2)
137
138     def test_subtraction(self):
139         """Test subtraction operation."""
140         self.assertEqual(calculator('subtract', 5, 3), 2)
141         self.assertEqual(calculator('subtract', 0, 1), -1)
142         self.assertEqual(calculator('subtract', -1, -1), 0)
143 if __name__ == '__main__':
144     unittest.main()
145

```

Output:

```

Assignment12.py
..
-----
Ran 2 tests in 0.000s

OK
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC>

```

Justification:

The program defines a calculator function that performs basic arithmetic operations based on the given operation. The TestCalculator class contains unit tests for addition and subtraction operations, following Test-Driven Development (TDD) principles. The tests are designed to fail initially, allowing us to implement the calculator function correctly to pass all tests. Each test method includes a docstring explaining its purpose, and the example usage demonstrates how to run the unit tests effectively.

Task Description -4 (Data Structures – Queue Implementation with AI Assistance)

➤ **Task:**

Use AI assistance to generate a Python program that implements a Queue data structure.

Instructions: Prompt AI to create a Queue class with the following methods:

- enqueue(element), dequeue(), front(), is_empty()
- Handle queue overflow and underflow conditions.
- Include appropriate docstrings for all methods.

Expected Output:

- A fully functional Queue implementation in Python.
- Proper error handling and documentation.

Prompt:

generate a python program to implement a Queue class with enqueue, dequeue, peek, isEmpty and size methods. Ensure that proper error handling for queue underflow and overflow conditions and include a docstring for each method to explain its functionality.

Code:

```

Assignment12.py
Lab > Assignment12.py > ...
148 # generate a python program to implement a Queue class with enqueue, dequeue, peek, isEmpty and size methods. Ensure that proper error handling for queue underflow and overflow conditions and include a docstring f
149 class Queue:
150     def __init__(self, max_size=10):
151         self.items = []
152         self.max_size = max_size
153
154     def enqueue(self, item):
155         """Add an item to the end of the queue."""
156         if len(self.items) >= self.max_size:
157             raise OverflowError("Queue overflow: Cannot enqueue to a full queue.")
158         self.items.append(item)
159
160     def dequeue(self):
161         """Remove and return the item at the front of the queue."""
162         if self.isEmpty():
163             raise IndexError("Queue underflow: Cannot dequeue from an empty queue.")
164         return self.items.pop(0)
165
166     def peek(self):
167         """Return the item at the front of the queue without removing it."""
168         if self.isEmpty():
169             raise IndexError("Queue is empty: Cannot peek at an empty queue.")
170         return self.items[0]
171
172     def isEmpty(self):
173         """Check if the queue is empty."""
174         return len(self.items) == 0
175
176     def size(self):
177         """Return the number of items in the queue."""
178         return len(self.items)
179
180 # Example usage:
181 if __name__ == "__main__":
182     queue = Queue(max_size=5)
183     queue.enqueue(1)
184     queue.enqueue(2)
185     queue.enqueue(3)
186     print("Front item:", queue.peek()) # Output: Front item: 1
187     print("Queue size:", queue.size()) # Output: Queue size: 3
188     print("Dequeued item:", queue.dequeue()) # Output: Dequeued item: 1
189     print("Is queue empty?", queue.isEmpty()) # Output: Is queue empty? False
190     queue.dequeue()
191     print("Is queue empty?", queue.isEmpty()) # Output: Is queue empty? True
192     try:
193         queue.dequeue() # This will raise an error
194     except IndexError as e:
195         print(e) # Output: Queue underflow: Cannot dequeue from an empty queue.
196     try:
197         queue.peek() # This will raise an error
198     except IndexError as e:
199         print(e) # Output: Queue is empty: Cannot peek at an empty queue.
200     try:
201         for i in range(6):
202             queue.enqueue(i) # This will raise an error on the 6th enqueue
203     except OverflowError as e:
204         print(e) # Output: Queue overflow: Cannot enqueue to a full queue.
205
206 queue.enqueue(2)
207 queue.enqueue(3)
208 print("Front item:", queue.peek()) # Output: Front item: 1
209 print("Queue size:", queue.size()) # Output: Queue size: 3
210 print("Dequeued item:", queue.dequeue()) # Output: Dequeued item: 1
211 print("Is queue empty?", queue.isEmpty()) # Output: Is queue empty? False
212 queue.dequeue()
213 queue.dequeue()
214 print("Is queue empty?", queue.isEmpty()) # Output: Is queue empty? True
215 try:
216     queue.dequeue() # This will raise an error
217 except IndexError as e:
218     print(e) # Output: Queue underflow: Cannot dequeue from an empty queue.
219 try:
220     queue.peek() # This will raise an error
221 except IndexError as e:
222     print(e) # Output: Queue is empty: Cannot peek at an empty queue.
223 try:
224     for i in range(6):
225         queue.enqueue(i) # This will raise an error on the 6th enqueue
226 except OverflowError as e:
227     print(e) # Output: Queue overflow: Cannot enqueue to a full queue.

```

Output:

```
Front item: 1
Queue size: 3
Dequeued item: 1
Is queue empty? False
Is queue empty? True
Queue underflow: Cannot dequeue from an empty queue.
Queue is empty: Cannot peek at an empty queue.
Queue overflow: Cannot enqueue to a full queue.
PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC> |
```

Justification:

The Queue class is implemented with methods for enqueueing, dequeuing, peeking, checking if the queue is empty, and getting the size of the queue. Each method includes error handling for queue overflow and underflow conditions. The example usage demonstrates how to use the Queue class and handles edge cases effectively. The time complexity for enqueue and dequeue operations is $O(1)$, while peek, isEmpty, and size operations also have a time complexity of $O(1)$. The space complexity is $O(n)$ where n is the number of items in the queue.

Task Description -5 (Algorithms – Bubble Sort vs Selection Sort)**➤ Task:**

Use AI to implement Bubble Sort & Selection Sort algorithm & compare their behavior.

Instructions:

bubble_sort(arr), selection_sort(arr)

- Include comments explaining each step.
- Add docstrings mentioning time and space complexity.

Expected Output:

- Correct Python implementations of both sorting algorithms.
- Complexity analysis in docstrings.

Prompt:

generate a python program to implement a bubble sort and selection sort algorithms and compare their performance their behavior with different input sizes. The program should include functions for both sorting methods, and it should allow to test both algorithms using different input sizes and measure their execution time. Additionally, the program should handle edge cases, such as sorting an empty list or a list with duplicate elements, and include docstring for each function, time and space complexity.

Code:

```

Assignment12.py X
Lab > Assignment12.py >...

207 """# Task-5
208 # generate a python program to implement a bubble sort and selection sort algorithms and compare their performance their behavior with different input sizes. The program should include functions for both sorting m
209 import time
210 def bubble_sort(arr):
211     """Sort the array using bubble sort algorithm."""
212     n = len(arr)
213     for i in range(n):
214         for j in range(0, n-i-1):
215             if arr[j] > arr[j+1]:
216                 arr[j], arr[j+1] = arr[j+1], arr[j]
217     return arr
218 def selection_sort(arr):
219     """Sort the array using selection sort algorithm."""
220     n = len(arr)
221     for i in range(n):
222         min_idx = i
223         for j in range(i+1, n):
224             if arr[j] < arr[min_idx]:
225                 min_idx = j
226         arr[i], arr[min_idx] = arr[min_idx], arr[i]
227     return arr
228 # Example usage:
229 if __name__ == "__main__":
230     # Test data
231     arr_sizes = [100, 1000, 5000]
232     for size in arr_sizes:
233         arr = list(range(size, 0, -1)) # Reverse sorted array
234         print(f"\nSorting array of size (size):")
235
236         # Test Bubble Sort
237         start_time = time.time()
238         bubble_sort(arr.copy())
239         end_time = time.time()
240         print(f"Bubble Sort: Time taken: (end_time - start_time:.6f) seconds")
241
242         # Test Selection Sort
243         start_time = time.time()
244         selection_sort(arr.copy())
245         end_time = time.time()
246         print(f"Selection Sort: Time taken: (end_time - start_time:.6f) seconds")
247
248     # Edge case: Sorting an empty list
249     empty_arr = []
250     print("\nSorting an empty list:")
251     print("Bubble Sort:", bubble_sort(empty_arr.copy())) # Output: []
252     print("Selection Sort:", selection_sort(empty_arr.copy())) # Output: []
253
254     # Edge case: Sorting a list with duplicate elements
255     duplicate_arr = [5, 3, 8, 3, 9, 1, 5]
256     print("\nSorting a list with duplicate elements:")
257     print("Bubble Sort:", bubble_sort(duplicate_arr.copy())) # Output: [1, 3, 3, 5, 5, 8, 9]
258     print("Selection Sort:", selection_sort(duplicate_arr.copy())) # Output: [1, 3, 3, 5, 5, 8, 9]
259

```

Output:

```

Sorting array of size 100:
Bubble Sort: Time taken: 0.000273 seconds
Selection Sort: Time taken: 0.000147 seconds

```

```

Sorting array of size 1000:
Bubble Sort: Time taken: 0.039545 seconds
Selection Sort: Time taken: 0.014833 seconds

```

```

Sorting array of size 5000:
Bubble Sort: Time taken: 1.275102 seconds
Selection Sort: Time taken: 0.485257 seconds

```

```

Sorting an empty list:
Bubble Sort: []
Selection Sort: []

```

```

Sorting a list with duplicate elements:
Bubble Sort: [1, 3, 3, 5, 5, 8, 9]
Selection Sort: [1, 3, 3, 5, 5, 8, 9]

```

```

PS C:\Users\bathi\OneDrive\Documents\Desktop\Python\AI_AC>

```

Justification:

The program implements both bubble sort and selection sort algorithms, allowing us to compare their performance with different input sizes. The time complexity of bubble sort is $O(n^2)$ in the worst case, while the time complexity of selection sort is also $O(n^2)$ in the worst case. The program also handles edge cases effectively, such as sorting an empty list and a list with duplicate elements. Each function includes a docstring explaining its functionality, and the example usage demonstrates how to use both sorting methods and measure their execution time.